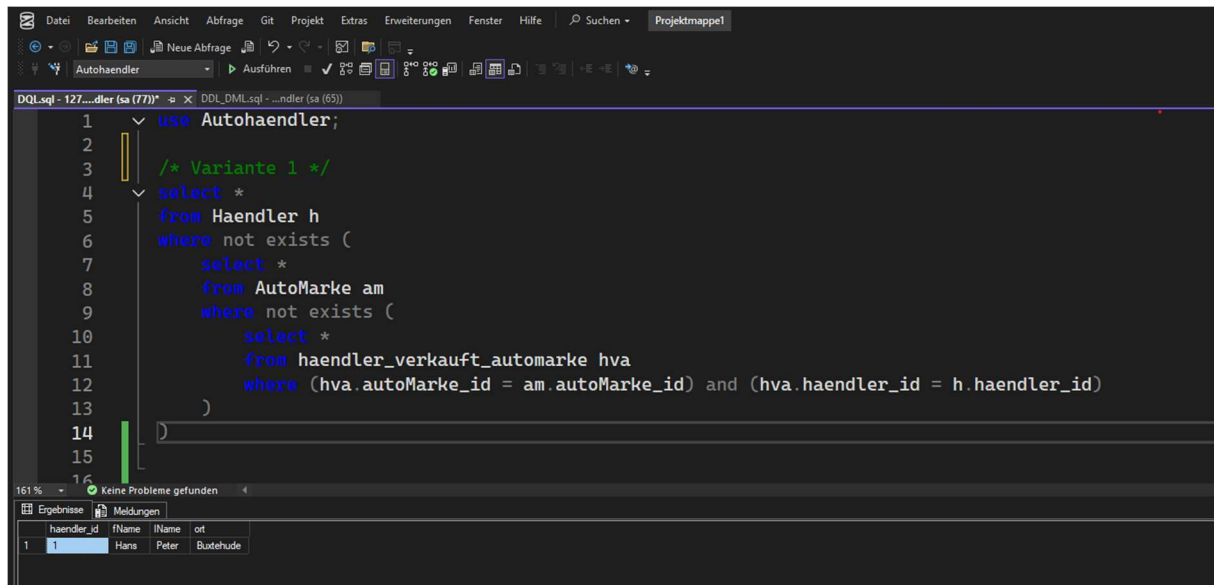


(creates, inserts .sql im Anhang)

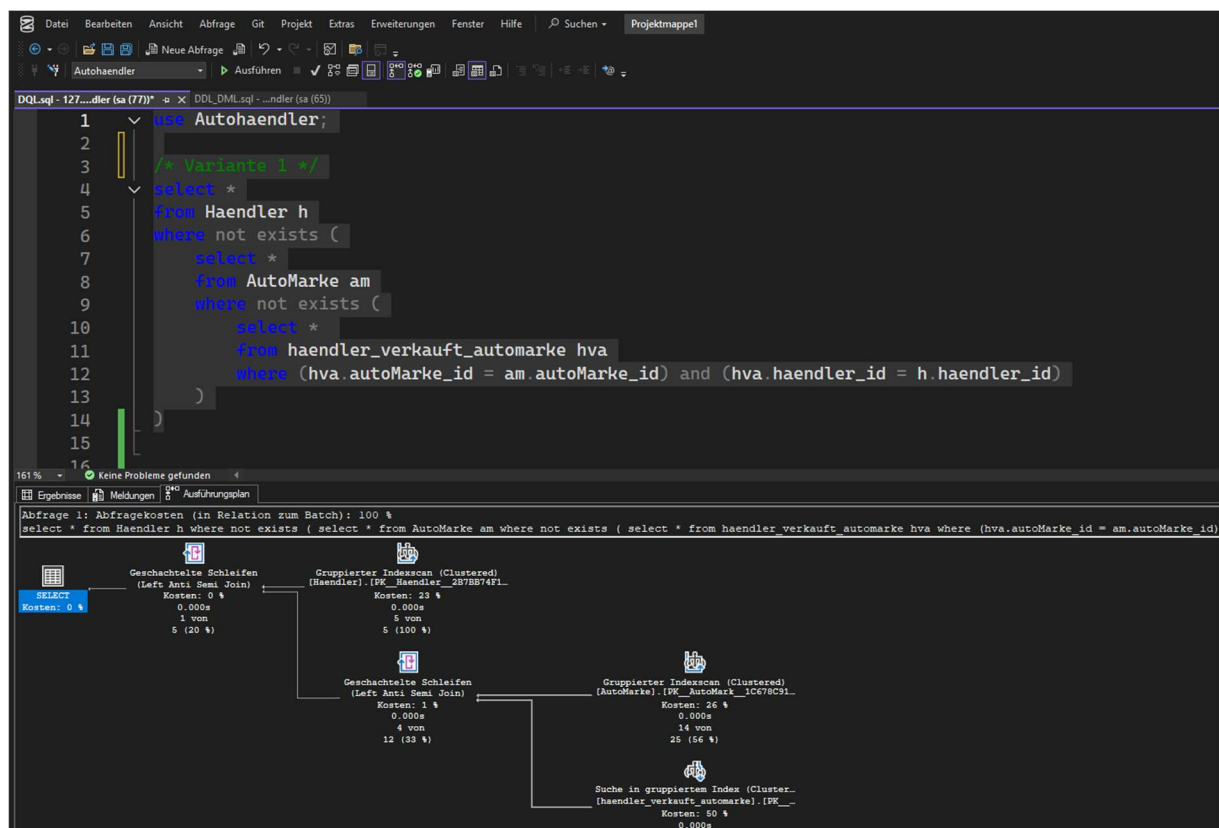
Variante 1)



The screenshot shows a SQL IDE window with a query editor and a results pane. The query is as follows:

```
1 use Autohaendler;
2
3 /* Variante 1 */
4 select *
5 from Haendler h
6 where not exists (
7     select *
8     from AutoMarke am
9     where not exists (
10         select *
11         from haendler_verkauft_automarke hva
12         where (hva.autoMarke_id = am.autoMarke_id) and (hva.haendler_id = h.haendler_id)
13     )
14 )
```

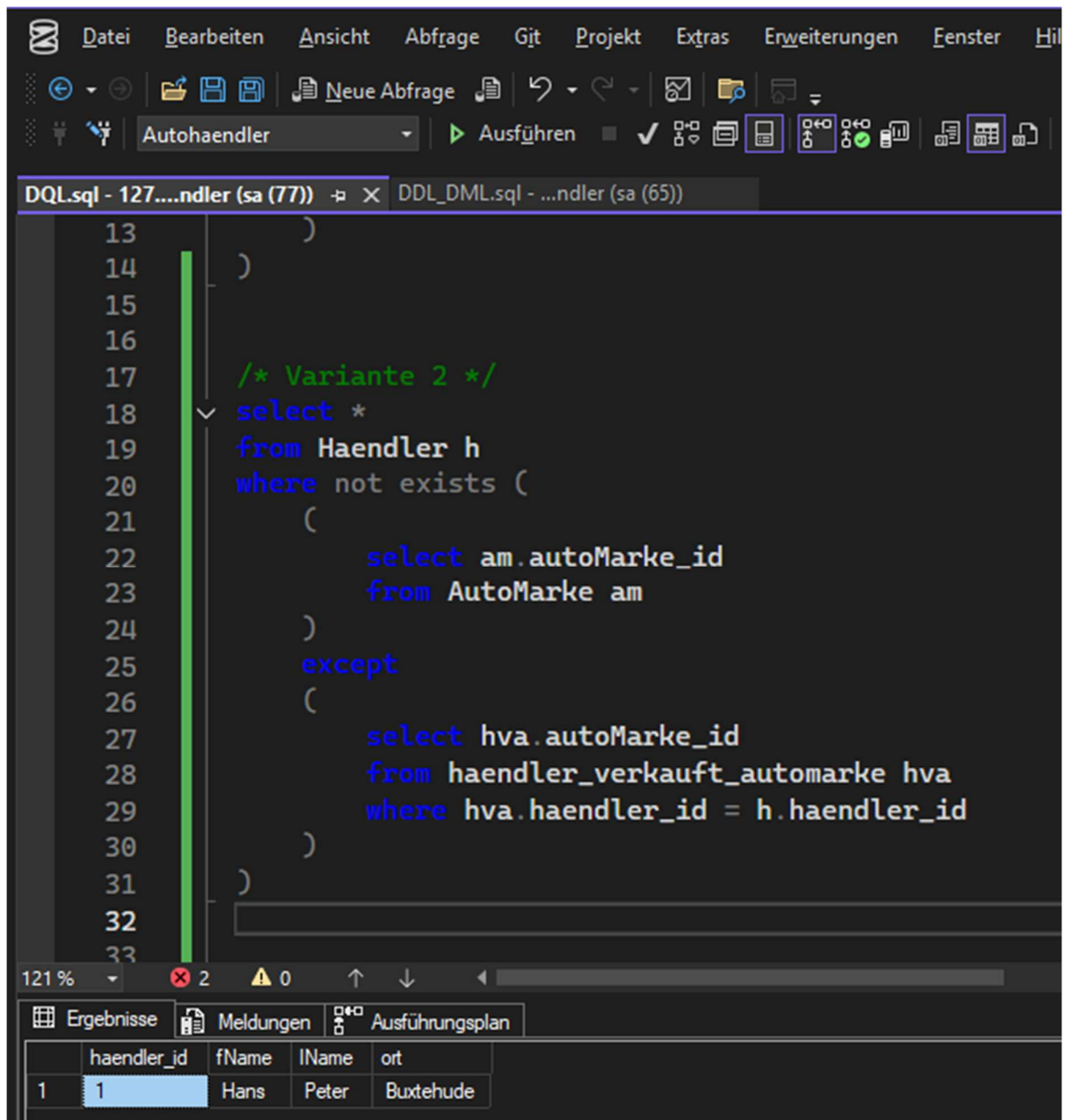
The results pane shows a table with columns: haendler_id, fName, lName, ort. The first row contains the values: 1, Hans, Peter, Budehude.



Erklärung Ausführungsplan:

1. Rechts unten:
in haendler_verkauft_automarke wird nach passenden autoMarke_id /
haendler_id gesucht.
2. Rechts Mitte:
alle Automarken aus AutoMarke auslesen.
3. Erstes Nested Loop (Anti Semi Join):
Prüft pro Automarke, ob kein Verkaufseintrag existiert (NOT EXISTS).
4. Links Mitte:
alle Händler aus Haendler auslesen.
5. Zweites Nested Loop (Anti Semi Join):
Prüft pro Händler, ob es keineAutomarke gibt, die von ihm nicht verkauft wird
6. Links oben (SELECT):
Gibt alle Händler zurück, die alle Marken verkaufen.

Variante 2)



```
13      )
14  )
15
16
17  /* Variante 2 */
18  select *
19  from Haendler h
20  where not exists (
21      (
22          select am.autoMarke_id
23          from AutoMarke am
24      )
25      except
26      (
27          select hva.autoMarke_id
28          from haendler_verkauft_automarke hva
29          where hva.haendler_id = h.haendler_id
30      )
31  )
32
33
```

121 % 2 0

Ergebnisse Meldungen Ausführungsplan

	haendler_id	fName	IName	ort
1	1	Hans	Peter	Buxtehude

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a SQL query in a dark-themed editor. The query is as follows:

```

13      )
14    )
15
16
17    /* Variante 2 */
18  select *
19  from Haendler h
20  where not exists (
21    (
22      select am.autoMarke_id
23      from AutoMarke am
24    )
25    except
26    (
27      select hva.autoMarke_id
28      from haendler_verkauft_automarke hva
29      where hva.haendler_id = h.haendler_id
30    )
31  )
32
33

```

The bottom pane shows the execution plan for the query. The plan is a tree diagram with the following components:

- SELECT**: Kosten: 0 %, 0.000s, 4 von 5 (20 %)
- Geschachtelte Schleifen (Left Anti Semi Join)**: Kosten: 0 %, 0.000s, 4 von 5 (20 %)
- Gruppierter Indexscan (Clustered) [Haendler].[FK_Haendler_257B574F1_...]**: Kosten: 23 %, 0.000s, 5 von 5 (100 %)
- Geschachtelte Schleifen (Left Anti Semi Join)**: Kosten: 1 %, 0.000s, 4 von 12 (33 %)
- Gruppierter Indexscan (Clustered) [AutoMarke].[PK_AutoMark_1C678C91_...]**: Kosten: 26 %, 0.000s, 14 von 25 (56 %)
- Suche in gruppiertem Index (Clustered) [haendler_verkaufte_automarke].[FK_...]**: Kosten: 50 %, 0.000s, 10 von 25 (40 %)

The status bar at the bottom indicates: "Die Abfrage wurde erfolgreich ausgeführt." and "127.0.0.1 (16.0 RTM) | sa (77) | Autohaendler | 00:00:00 | 1 Zeilen".

Erklärung Ausführungsplan:

1. Rechts unten:
in haendler_verkauft_automarke alle Automarken, die ein Händler verkauft werden suchen.
2. Rechts Mitte:
alle Automarken aus AutoMarke auslesen.

3. Inneres Nested Loop (Anti Semi Join):
Hier werden beide Mengen verglichen und dann das EXCEPT gebildet → Marken, finden die der Händler nicht verkauft.
4. Links Mitte:
alle Händler aus Haendler auslesen.
5. Äußeres Nested Loop (Anti Semi Join):
pro Händler prüfen, ob das EXCEPT-Ergebnis leer ist → also welcher Händler verkauft alle Marken.
6. Links oben (SELECT):
Gibt genau diese Händler aus.

Variante 3)

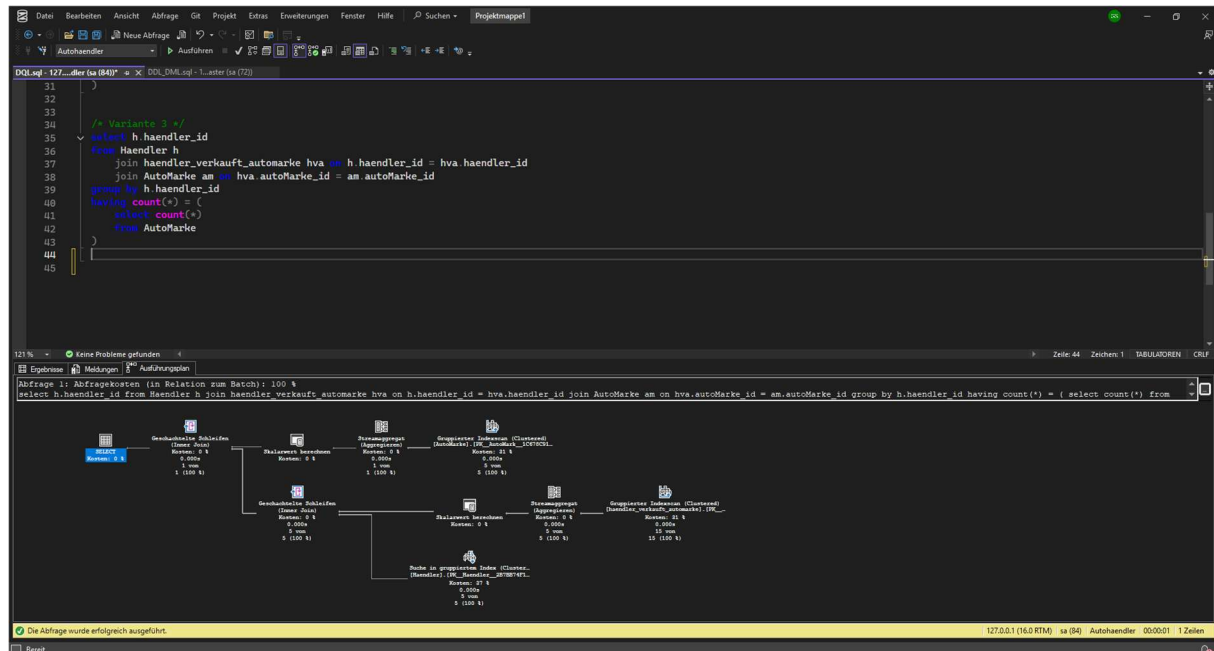
The screenshot displays the Microsoft SQL Server Enterprise Manager interface. The left pane shows the 'Objekt-Explorer' with a tree view of the server '127.0.0.1 (SQL Server 16.0.4210.1 - sa)'. The right pane shows a T-SQL query in the 'DQLsql - 127...dler (sa (84))' window. The query is as follows:

```
28      from haendler_verkauft_automarke hva
29      where hva.haendler_id = h.haendler_id
30    )
31  )
32
33  /* Variante 3 */
34
35  select h.haendler_id
36  from Haendler h
37  join haendler_verkauft_automarke hva on h.haendler_id = hva.haendler_id
38  join AutoMarke am on hva.autoMarke_id = am.autoMarke_id
39  group by h.haendler_id
40  having count(*) = (
41    select count(*)
42    from AutoMarke
43  )
44
45
```

Below the query editor, the 'Ergebnisse' pane shows the results of the query. The results are displayed in a table with one column, 'haendler_id', and one row with the value '1'.

haendler_id
1

A status bar at the bottom of the window indicates 'Die Abfrage wurde erfolgreich ausgeführt.' (The query was successfully executed).



Erklärung Ausführungsplan:

1. Ganz rechts unten:
alle Datensätze aus haendler_verkauft_automarke auslesen.
2. Rechts Mitte:
hier AutoMarke mit oberer Ausgabe verbinden, um Markeninformationen zu holen (also der Join).
3. Links Mitte:
nach der haendler_id gruppieren, jetzt zählen wie viele Marken jeder Händler verkauft.
4. Unterer Zweig (Subquery):
alle vorhandenen Marken aus AutoMarke zählen.
5. Vergleich (Scalar Compare):
vergleichen von dem wie viel der Händler verkauft zu dem wie viele Marken es insgesamt gibt.
6. Links oben (SELECT):
alle Händler zurückgeben die alle Automarken verkaufen.

Variante 4)

The screenshot shows a SQL IDE with a menu bar (Datei, Bearbeiten, Ansicht, Abfrage, Git, Projekt, Extras, Erweiterungen, Fenster, Hilfe, Suchen) and a toolbar. The main editor displays a SQL query for 'Variante 4*'. The query is as follows:

```
44  
45  
46  /* Variante 4*/  
47  (  
48      select h.haendler_id  
49      from Haendler h  
50  )  
51  except  
52  (  
53      select inner_erg.haendler_id  
54      from (  
55          (  
56              select inner_h.haendler_id, am.autoMarke_id  
57              from Haendler inner_h, AutoMarke am  
58          )  
59          except  
60          (  
61              select hva.haendler_id, hva.autoMarke_id  
62              from haendler_verkauft_automarke hva  
63          )  
64      ) inner_erg  
65  )  
66
```

The status bar at the bottom indicates '121 %', 'Keine Probleme gefunden', and 'Zeile: 66'. Below the editor, the 'Ergebnisse' (Results) tab is active, showing a table with one column 'haendler_id' and one row with the value '1'.

haendler_id
1

A yellow status bar at the bottom of the IDE displays the message: 'Die Abfrage wurde erfolgreich ausgeführt.' (The query was successfully executed.) followed by '127.0.0.1 (16.0 RTM) | sa (64) | Aut'.

At the very bottom, a message box says 'Element(e) gespeichert' (Element(s) saved).

The screenshot shows a SQL IDE with a query window and an execution plan window. The query is as follows:

```

46  /* Variante 4 */
47  (
48    select h.haendler_id
49    from Haendler h
50  )
51  except
52  (
53    select inner_erg.haendler_id
54    from (
55      (
56        select inner_h.haendler_id, am.autoMarke_id
57        from Haendler inner_h, AutoMarke am
58      )
59      except
60      (
61        select hva.haendler_id, hva.autoMarke_id
62        from haendler_verkauft_automarke hva
63      )
64    ) inner_erg
65  )

```

The execution plan shows the following steps:

- Abfrage 1: Abfragekosten (in Relation zum Batch): 100 %**
 (select h.haendler_id from Haendler h) except (select inner_erg.haendler_id from ((select inner_h.haendler_id, am.autoMarke_id from Haendler inner_h, AutoMarke am) ex
- Geschachtelte Schleifen (Left Anti Semi Join)**
 Kosten: 0 %
 0.000s
 1 von 3 (33 %)
- Gruppierter Indexscan (Clustered)**
 [Haendler].[PK_Haendler_2B7BB74F1]
 Kosten: 18 %
 0.000s
 5 von 5 (100 %)
- Geschachtelte Schleifen (Left Anti Semi Join)**
 Kosten: 1 %
 0.000s
 4 von 5 (80 %)
- Geschachtelte Schleifen (Inner Join)**
 Kosten: 1 %
 0.000s
 14 von 25 (56 %)
- Suche in gruppiertem Index (Clustered)**
 [Haendler].[PK_Haendler_2B7BB74F1]
 Kosten: 22 %
 0.000s
 5 von 5 (100 %)
- Suche in gruppiertem Index (Clustered)**
 [haendler_verkauft_automarke].[PK_]
 Kosten: 39 %
 0.000s
 10 von 25 (40 %)
- Gruppierter Indexscan (Clustered)**
 [AutoMarke].[PK_AutoMark_1C678C91]
 Kosten: 20 %
 0.000s
 14 von 25 (56 %)

The status bar at the bottom indicates: "Die Abfrage wurde erfolgreich ausgeführt." and "Bereit".

Erklärung Ausführungsplan:

1. AutoMarke (Clustered Index Scan):
alle Marken auslesen
2. Haendler (inner_h) (Index Seek):
die Händler laden und mit einem sog. Nested Loop (also im Prinzip der Inner Join) mit allen Marken kreuzen (also cross joinen)
3. haendler_verkauft_automarke (Index Seek):
vorhandene Paare holen
4. Nested Loop (Left Anti Semi Join):
cross join aus Händler und Automarke EXCEPT der verkauften Paare → ergibt die somit „fehlende Paare“ pro Händler, also kriegt man so die Paare die er nicht verkauft

5. Haendler (Index Seek):
die komplette Händlerliste für die Außenseite auslesen.
6. Left Anti Semi Join + SELECT:
alle Händler minus der Händler mit „fehlenden Paaren“ → übrig bleiben Händler,
die alle Marken verkaufen.